```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
image_path = "/content/drogon2.jpg"

original_image = cv2.imread(image_path)

if original_image is None:
    print("Error: Image not found.")
else:
    print("Image loaded successfully!")
    print("Image shape:", original_image.shape)
```

```
Image loaded successfully!
Image shape: (260, 520, 3)
```

```
rgb_image = cv2.cvtColor(
    original_image,
    cv2.COLOR_BGR2RGB
)
```

```
plt.figure(figsize=(10, 10))

plt.imshow(rgb_image)

plt.title("Original Dragon Image")

plt.axis("off")

plt.show()
```


```
gray_image = cv2.cvtColor(
    original_image,
    cv2.COLOR_BGR2GRAY
)

gray_image_float = np.float32(gray_image)
```

```
plt.figure(figsize=(10, 10))

plt.imshow(
    gray_image,
    cmap="gray"
)

plt.title("Grayscale Dragon Image")

plt.axis("off")

plt.show()
```


```
sobelx = cv2.Sobel(
    gray_image_float,
    cv2.CV_32F,
    1,
    0,
    ksize=3
)

print("Sobel X calculated")
```

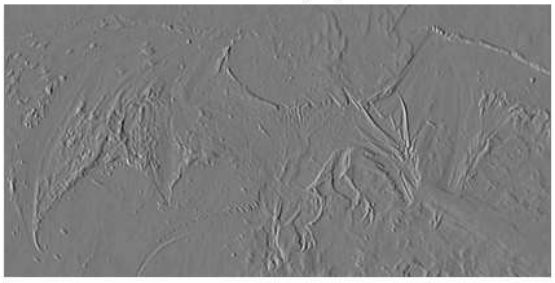
```
Sobel X calculated
```

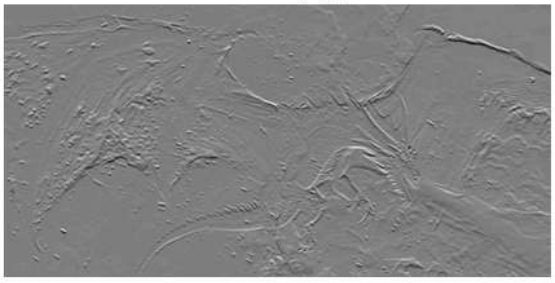
```
sobely = cv2.Sobel(
    gray_image_float,
    cv2.CV_32F,
    0,
    1,
    ksize=3
)

print("Sobel Y calculated")
```

```
Sobel Y calculated
```

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
```

```
plt.imshow(
    sobelx,
    cmap="gray"
)

plt.title("Sobel X (Gx)")

plt.axis("off")


plt.subplot(1, 2, 2)

plt.imshow(
    sobely,
    cmap="gray"
)

plt.title("Sobel Y (Gy)")

plt.axis("off")

plt.show()
```






```
harris_response = cv2.cornerHarris(
    gray_image_float,
    blockSize=2,
    ksize=3,
    k=0.04
)

print(
    "Harris Corner Response Min:",
    harris_response.min()
)

print(
    "Harris Corner Response Max:",
```

```
    harris_response.max()
)
```

```
Harris Corner Response Min: -41590384.0
Harris Corner Response Max: 154482030.0
```

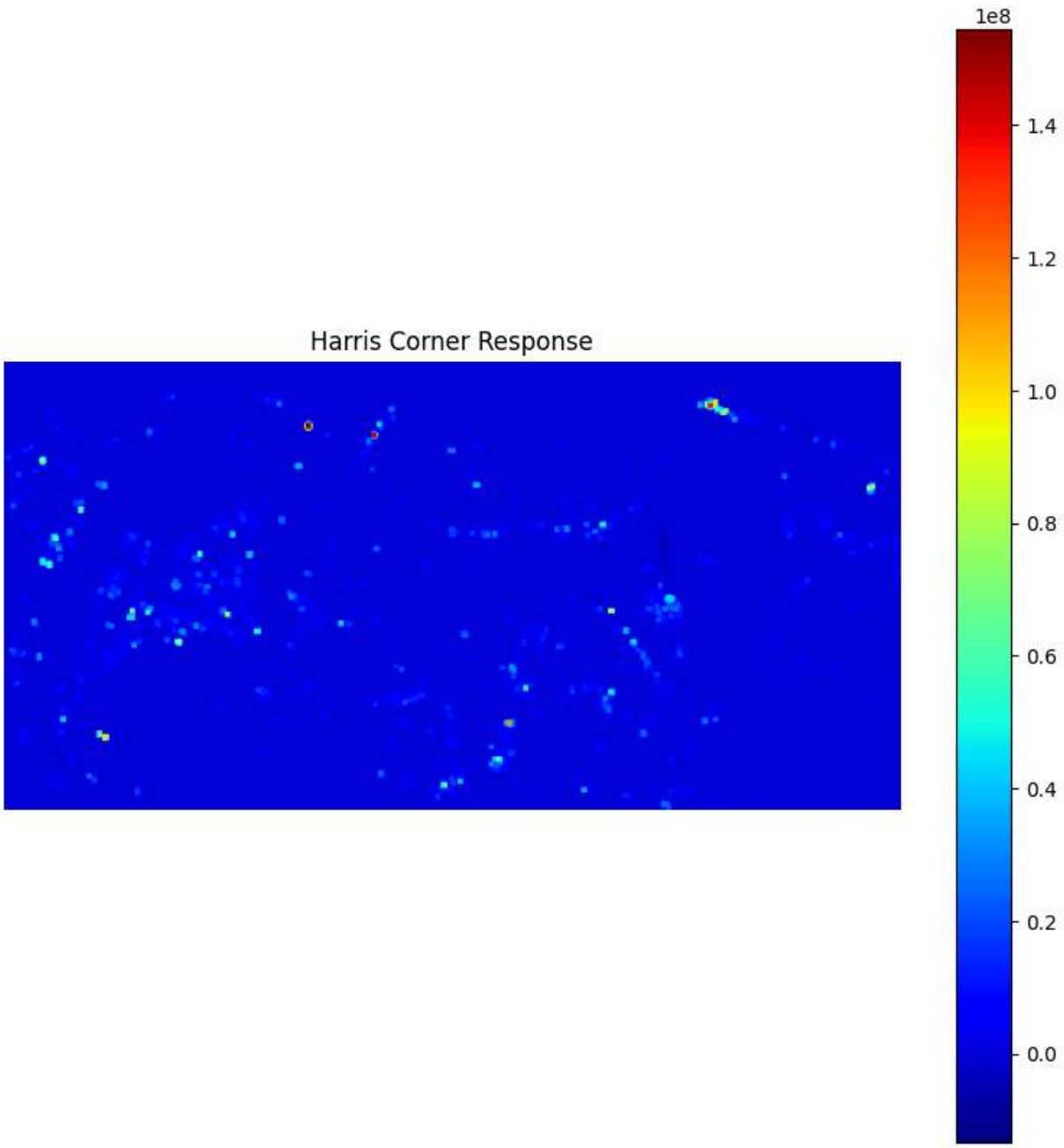
```
harris_response = cv2.dilate(
    harris_response,
    None
)

print("Harris response dilated successfully")
```

```
Harris response dilated successfully
```

```
corners_image = rgb_image.copy()

threshold = 0.01 * harris_response.max()

corners_image[
    harris_response > threshold
] = [255, 0, 0]
```

```
plt.figure(figsize=(10, 10))

plt.imshow(corners_image)

plt.title(
    "Harris Corners Detected on Dragon Image"
)

plt.axis("off")

plt.show()
```


```
plt.figure(figsize=(10, 10))

plt.imshow(
    harris_response,
    cmap="jet"
)

plt.colorbar()

plt.title("Harris Corner Response")

plt.axis("off")

plt.show()
```



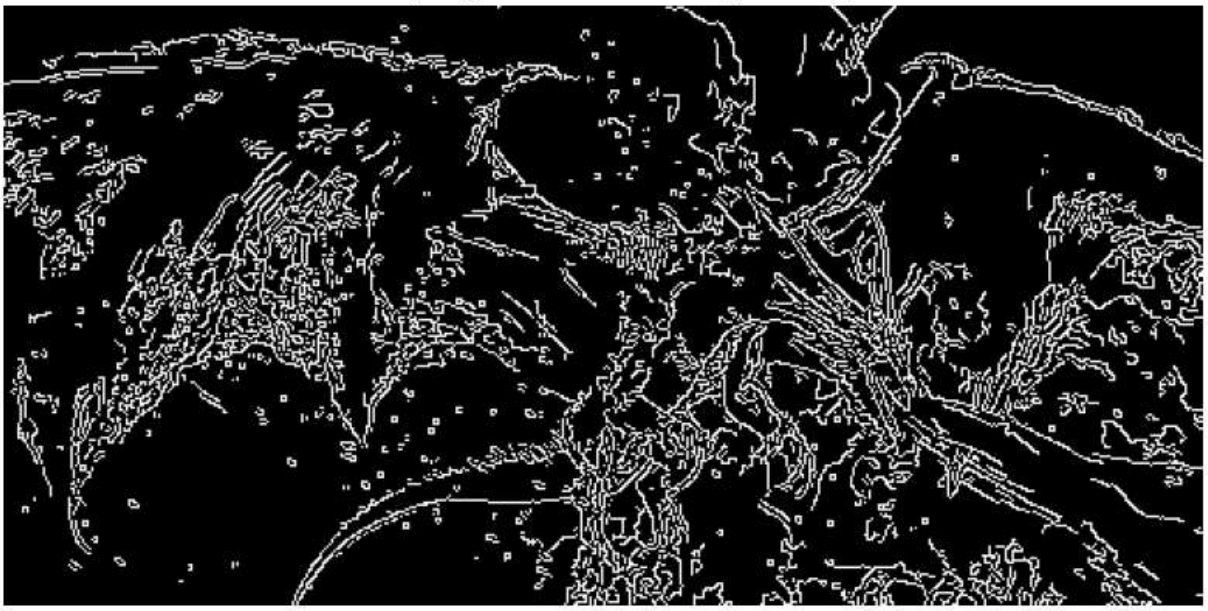

```
gray_image_uint8 = np.uint8(gray_image_float)

print("Image converted to uint8")
```

```
Image converted to uint8
```

```
edges_canny = cv2.Canny(
    gray_image_uint8,
    threshold1=100,
    threshold2=200
)

print(
    "Canny Edge Map Min:",
    edges_canny.min()
)

print(
    "Canny Edge Map Max:",
    edges_canny.max()
)
```

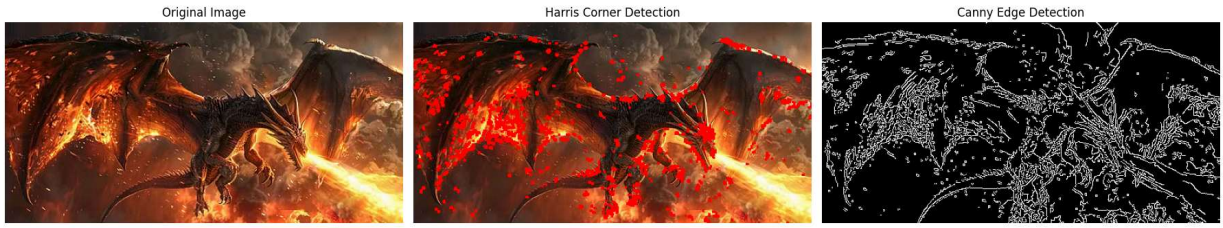
```
Canny Edge Map Min: 0
Canny Edge Map Max: 255
```

```
plt.figure(figsize=(10, 10))

plt.imshow(
    edges_canny,
    cmap="gray"
)

plt.title(
    "Canny Edge Detection for Dragon Image"
)

plt.axis("off")

plt.show()
```




```
plt.figure(figsize=(18, 6))

# Original
plt.subplot(1, 3, 1)

plt.imshow(rgb_image)

plt.title("Original Image")

plt.axis("off")


# Harris
plt.subplot(1, 3, 2)

plt.imshow(corners_image)

plt.title("Harris Corner Detection")

plt.axis("off")


# Canny
plt.subplot(1, 3, 3)

plt.imshow(
    edges_canny,
```

```
    cmap="gray"
)

plt.title("Canny Edge Detection")

plt.axis("off")


plt.tight_layout()

plt.show()
```




```
# Original Image
plt.figure(figsize=(12, 12))
plt.imshow(rgb_image)
plt.title("Original Dragon Image", fontsize=20)
plt.axis("off")
plt.show()

# Harris Corner Detection
plt.figure(figsize=(12, 12))
plt.imshow(corners_image)
plt.title("Harris Corner Detection", fontsize=20)
plt.axis("off")
plt.show()

# Canny Edge Detection
plt.figure(figsize=(12, 12))
plt.imshow(edges_canny, cmap="gray")
plt.title("Canny Edge Detection", fontsize=20)
plt.axis("off")
plt.show()
```